*Last updated on **May 11, 2026***

# What is a Genie Space

This page introduces Genie Spaces, a Databricks feature that allows business teams to interact with their data using natural language. It uses generative AI tailored to your organization's terminology and data, with the ability to monitor and refine its performance through user feedback.

## Overview

Domain experts, such as data analysts, configure Genie Spaces with datasets, sample queries, and text guidelines to help Genie translate business questions into analytical queries. After setup, business users can ask questions and generate visualizations to understand operational data. You can continuously update Genie's semantic knowledge as your data changes and users pose new questions. For additional information about Databricks AI-powered features, see Databricks AI assistive features.

Genie selects relevant names and descriptions from annotated tables and columns to convert natural language questions to an equivalent SQL query. Then, it responds with the generated query and results table, if possible. If Genie can't generate an answer, it can ask follow-up questions to clarify before providing a response.

## Example use cases

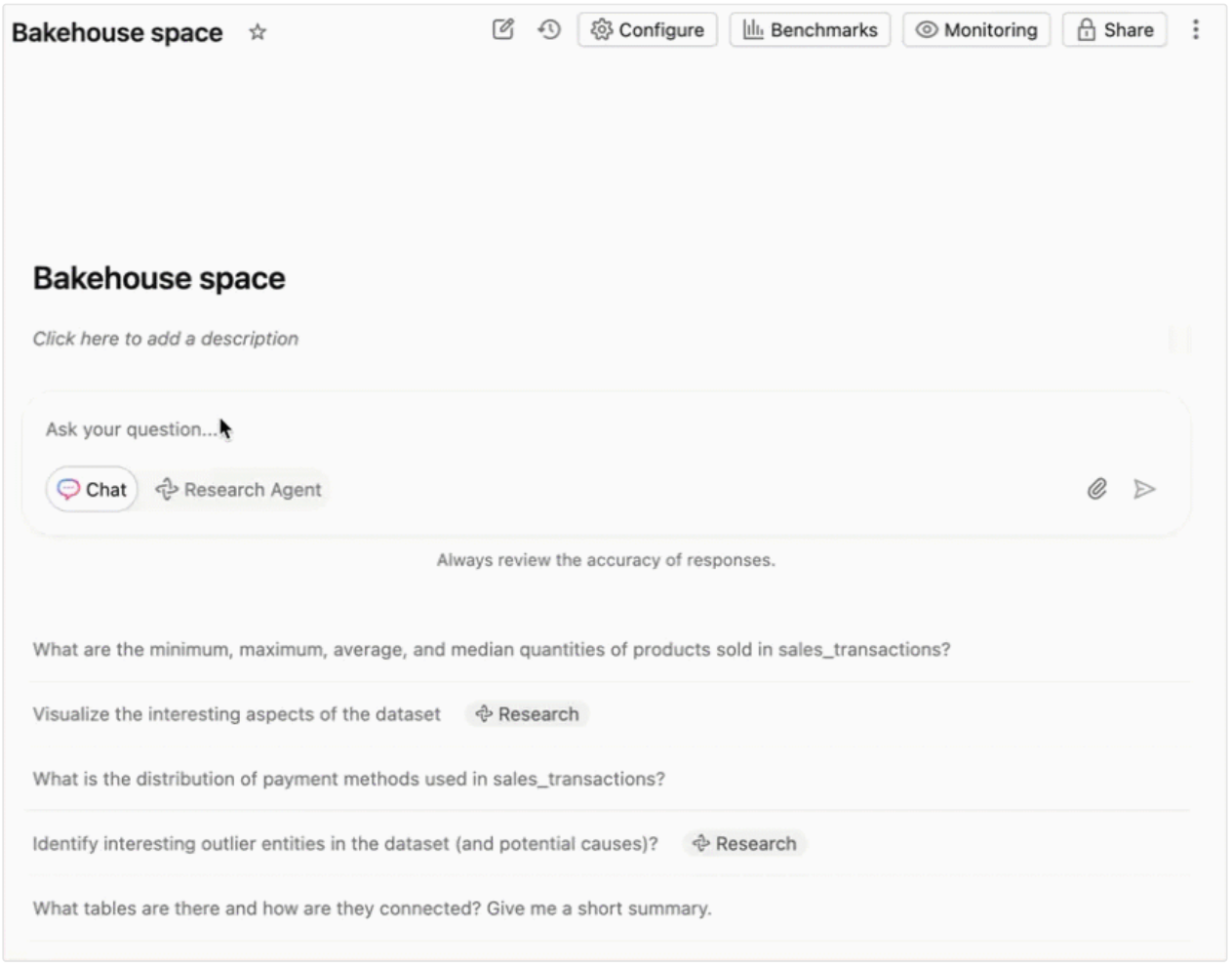
You can create different Genie Spaces to serve various non-technical audiences. The following scenarios describe two possible use cases.

### Example 1: Visualize top selling product

A sales manager wants to understand the top selling product over time i
They can interact with the Genie Space using natural language and auton

generate a visualization.

The following GIF shows this interaction:




## Example 2: Tracking logistics

A logistics company wants to use Genie Spaces to help business users from different departments track operational and financial details. They set up a Genie Space for their shipment facility managers to track shipments and another for their financial executives to understand their financial health.

# What data should I use?

A Genie Space is based on data registered to Unity Catalog, including managed tables, external tables, foreign tables, views, metric views, and materialized views. Genie uses the metadata attached to Unity Catalog objects, as well as an author-curated space-level knowledge store, to generate responses. Well-annotated datasets, paired with

specific instructions that you provide, are key to creating a positive experience for end users.

> **NOTE**
>
> Genie works with **structured data only**. It cannot answer questions about unstructured data such as PDFs, Word documents, or other file-based content. To give Genie access to unstructured documents, use Chat in Genie, which can connect to external document sources such as Google Drive or SharePoint.

## File uploads

> **PREVIEW**
>
> This feature is in Public Preview.

File uploads allow users to blend their local CSV and Excel files with Unity Catalog data to answer questions. To enable file uploads, contact your Databricks account team. For more information, see Upload a file.

# How Genie Spaces works

Genie Spaces use a compound AI system to interpret business questions and generate answers. Instead of using a single large language model, compound AI systems process tasks in AI applications by combining multiple interacting components. Compound AI systems are an increasingly common design pattern for AI applications because of their performance and flexibility. For more information, see The Shift from Models to Compound AI Systems.

# Language support

You can use Genie Spaces in languages other than English, such as Portuguese and French. However, the underlying agent framework wraps prompts in English.

Databricks recommends that space creators add as much metadata as possible in their language of choice. Genie responses might sometimes appear in English due to

the underlying system prompts.

# What is a Genie Space knowledge store?

The Genie Space knowledge store allows authors to:

- **Edit metadata locally:** Genie authors can add space-specific metadata to data assets. For example, it can include company-specific information relevant to how the space is used. This includes table and column metadata descriptions, column-level synonyms, and prompt matching capabilities, which Genie consults when generating answers. A detailed metadata layer helps Genie retrieve the correct information and produce more accurate results.
- **Provide structured, fine-grained instructions:** Authors can define JOIN relationships between tables, to teach Genie how to author SQL across multiple tables.

See Tune Genie Space quality.

# How do Genie Spaces generate a response?

When a user submits a question, Genie parses the request, identifies relevant data sources, and determines how to generate an appropriate response. Details provided by authors, combined with relevant Unity Catalog comments, metadata, and sample values from selected columns, allow Genie to infer both business and technical logic. For more information, see Databricks AI assistive features trust and safety and prompt matching. Genie intelligently filters example SQL queries, table and column metadata, and chat history to select the most relevant context for answering the request.

Genie generates responses using components such as the following:

- **Unity Catalog table metadata**: Includes table names, descriptions, and defined primary key (PK) and foreign key (FK) relationships. Genie uses this data as it parses the request and converts the natural language prompt to SQL.

- **Column names and descriptions**: Genie intelligently filters for relevant column names and descriptions to include.
- **Knowledge store context**: Authors can locally edit asset metadata and choose columns that provide relevant values to Genie. This helps Genie generate more accurate responses and doesn't alter existing Unity Catalog metadata. See Build a knowledge store.
- **Example SQL queries**: Genie intelligently selects relevant SQL examples from **SQL Queries**.
- **SQL functions**: All SQL functions that have been added in the space.
- **Instructions**: The plain-text notes provided as **General instructions** are included as context.
- **Prompt and responses history**: Prompts and responses from the current chat are included as context. If necessary, because of set token limits, the oldest parts of the chat record are excluded.

> **NOTE**
>
> Some table details, such as the owner and table size, are not included by default. To access this information, use views from the information schema available for all Unity Catalog catalogs. Default views might include unnecessary details, so creating a custom view on top of that can help focus on the specific information you need. For more information about what's available in the information schema, see Information schema.

In many cases, Genie generates a SQL query that runs on the space's SQL warehouse. Generated queries are always read-only. Retries are handled automatically, and the SQL warehouse handles concurrency and scale. The result set is presented as part of the response.

Genie maintains strong security and privacy controls. For details, see Databricks AI assistive features trust and safety.

# Improve response accuracy using Inspect

**PREVIEW**

This feature is in Public Preview.

Inspect uses advanced reasoning to review and improve the accuracy of Genie's generated SQL queries. When you enable Inspect for a response, Genie:

1. Reviews the initially generated SQL query.
2. Authors smaller SQL statements to verify specific aspects of the query, such as:
   - Confirming the correct filter values are included.
   - Validating date range logic, such as trailing 7-day windows.
   - Checking join conditions and aggregations.
3. Identifies gaps or potential issues in the original query.
4. If issues are identified, generates an improved SQL query that resolves them.
5. Performs a final comparison between the original and improved queries.
6. Returns the query that most accurately answers your question.

Use Inspect when you want additional confidence in query accuracy, especially for complex queries involving filters, date ranges, or multiple tables.

# Set up a Genie Space

You can create a Genie Space if you have the following:

- The Databricks SQL entitlement.
- At least CAN USE permission on a pro or serverless SQL warehouse.
- At least `SELECT` privileges on one or more Unity Catalog data objects.

See Create and manage a Genie Space.

## Companion Genie Spaces for AI/BI dashboards (Public Preview)

You can use natural language prompts to generate visualizations for AI/BI dashboards with Genie Code. See Use Genie Code for dashboard authoring.

When you create a dashboard, Databricks automatically creates a companion Genie Space that allows business users to conduct self-serve data analytics using natural language. See Genie Spaces with dashboards.

# Interact with a Genie Space

Business teams are the end users for a Genie Space. To use a Genie Space, business users must have:

- The consumer access or Databricks SQL entitlement.
- At least `SELECT` privileges on all of the Unity Catalog data objects used in the space. Users only see data they have permission to access.

Queries run using the compute credentials embedded by the author who configured the warehouse. End users do not need direct warehouse permissions.

Business users can help curate a space by testing it and providing feedback during development. To learn more about how business users can start working with a Genie Space, see Use a Genie Space to explore business data.

# Trusted assets

Trusted assets convey an extra layer of assurance in the accuracy of a result to a space user. When the exact text of a parameterized example query or SQL function is used to generate a response, Genie marks the response as **Trusted**. See Trusted assets to learn more about trusted assets and working with parameterized queries.

# Evaluate responses with benchmarks

Benchmarks allow you to scale up testing and evaluation of individual responses in a Genie Space. Unlike instructions, benchmarks are meant to evaluate, not inform, your Genie Space. Genie does not use benchmark questions or example SQL to improve Genie's context.

Using benchmarks, you can run a collection of test questions and use the responses to measure Genie's accuracy. Optionally, you can include a SQL statement that returns

the expected results. When the benchmark question runs, Genie's response is compared to the results provided by the SQL statement and scored for accuracy. The question is marked for review if no SQL answer has been provided.

See Benchmarks.

# How data access works

Data access in a Genie Space is governed by Unity Catalog. When a user asks a question, the generated SQL query runs against the data using the compute credentials embedded by the space author (the configured SQL warehouse). Each user's own Unity Catalog data permissions are applied to the query results. Users only see data they are authorized to access. Any question about data they cannot access returns an empty response.

This means:

- You do not need to grant users direct warehouse permissions.
- Row filters and column masks defined in Unity Catalog are automatically enforced per user.
- To implement per-user data filtering, apply row-level security to the underlying tables in Unity Catalog. See Row filters and column masks.

For information about setting up user permissions for a Genie Space, see Share a Genie Space.

# Privacy and security

Data access in a Genie Space is governed by Unity Catalog, including any row filters and column masks that have been applied to your tables. See Data access control and Row filters and column masks.

For other privacy and security FAQs, see the privacy and security FAQs for AI assistive features.

# Additional resources

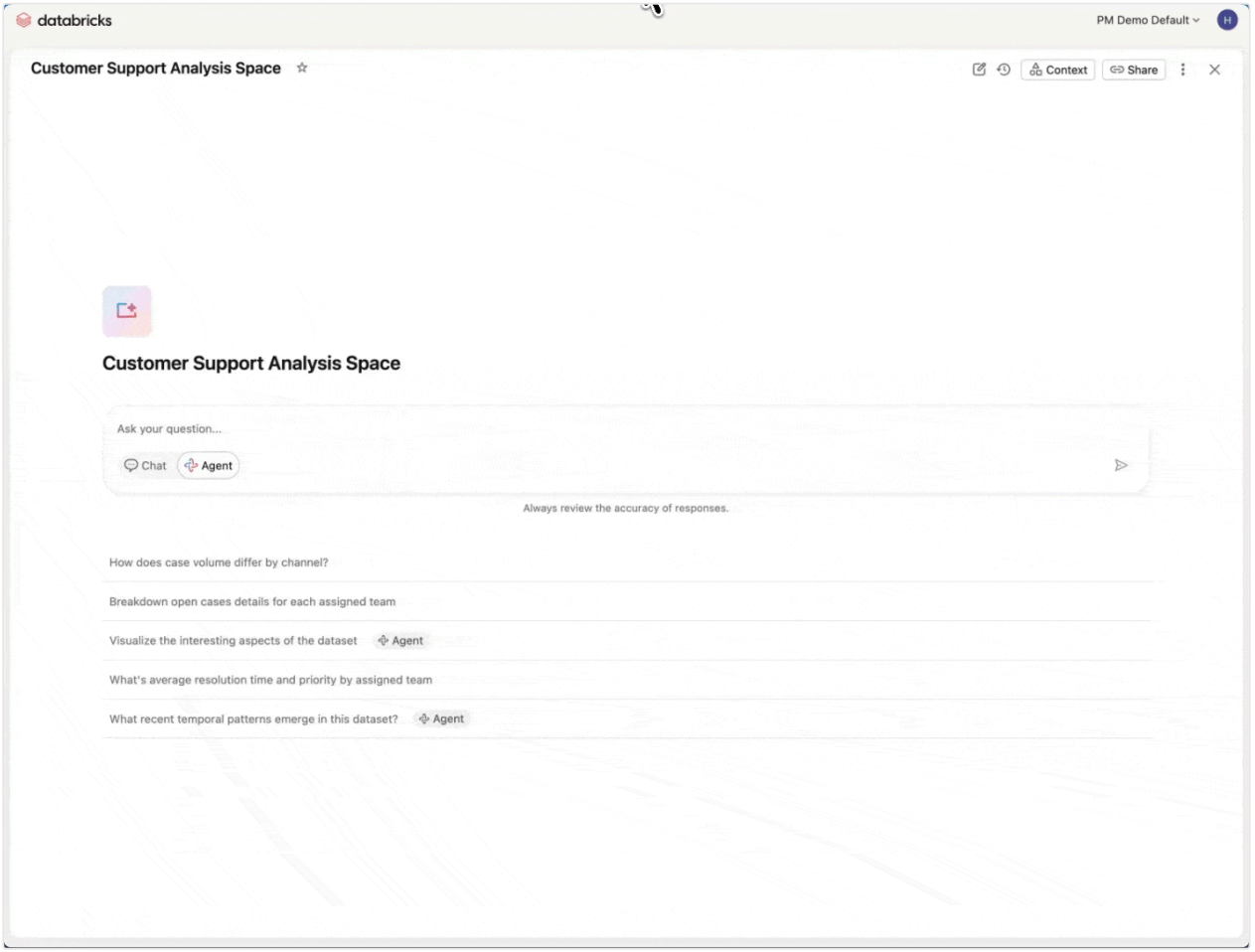
- To use the Genie API to integrate Genie into applications, chatbots, and agent frameworks, see Use the Genie Spaces API.
- To use audit logs to track Genie activity and usage, see Audit logs for AI/BI.
- For best practices and troubleshooting, see Curate an effective Genie Space.

*Last updated on **May 11, 2026***

# Agent mode in Genie Spaces

> **NOTE**
>
> Agent mode was formerly known as Research Agent.

Agent mode extends Genie's capabilities to answer both straightforward data questions and complex business questions. It uses multi-step reasoning and hypothesis testing to uncover deeper insights. When you ask a question, Agent mode creates and refines a research plan, running multiple SQL queries, learning from each result, and iterating until it has enough evidence to provide a comprehensive answer.




Unlike standard Genie queries, Agent mode:

Ask Genie

- **Creates research plans**: Develops a structured approach and hypotheses for answering complex questions.
- **Runs multiple queries**: Executes several SQL queries to gather evidence from different angles.
- **Learns and iterates**: Continuously adjusts its approach based on what it discovers, refining its reasoning until it's confident in the answer.
- **Delivers comprehensive reports**: Provides detailed summaries with citations, visualizations, and supporting tables.

Agent mode is designed for both complex, exploratory questions and straightforward data questions like:

- What are our top-selling products this quarter?
- What customer segments are most profitable?
- Why did revenue spike in June 2025?
- How can we reduce order cancellations this holiday season?
- What factors contributed to customer churn last quarter?
- Which marketing campaigns had the best ROI and why?

# Requirements and availability

Agent mode is in Public Preview. Workspace admins control access using the previews page. See Manage workspace-level previews.

To use Agent mode, your workspace must meet the following criteria.

## Have a well-prepared Genie Space

Agent mode relies heavily on the context within your Genie Space to reason about your data. You must meet all requirements for setting up a Genie Space. See Set up a Genie Space.

To optimize your Genie Space for Agent mode, follow the comprehensive guidance in:

- Create and manage a Genie Space: Initial setup and configuration

- Curate an effective Genie Space: Best practices for setting up an effective Genie Space
- Tune Genie Space quality: Adding SQL examples, instructions, and knowledge store context

## Regional availability

| Region | Availability |
|---|---|
| Americas & Europe workspaces | Available (cross-Geo not required) |
| Australia, New Zealand, and Japan workspaces | Available (cross-Geo not required) |
| All other regions | Available only if cross-Geo is enabled. See Enable cross-Geo processing. |

# Manage Agent mode

Workspace admins can control access to **Genie Agent mode (Public Preview)** using the previews page. See Manage workspace-level previews.

After Agent mode is enabled, it is available for all Genie Spaces within that workspace. All users with access to a Genie Space can use Agent mode. All requirements for accessing a Genie Space and its data apply. See Set up a Genie Space.

# Use Agent mode

To ask questions using Agent mode:

1. Open a Genie Space.
2. Click the Agent mode icon in the chat box.
3. Enter a question and send your prompt. Agent mode begins its research and may ask follow-up questions to clarify your intent. Agent mode responses may take longer to generate than standard Genie Space responses. When complete, it

answers with a final report containing specific findings, visualizations, and citations to research steps.

4. After the report is generated, you can send additional clarification prompts to refine your report or generate new reports on different research questions. You can provide feedback to help improve Agent mode.

# Export Agent mode reports

You can export Agent mode reports as PDF files for sharing or offline review. After Agent mode completes a report, click **Download PDF** at the bottom right of the report to download it, including findings, visualizations, and citations.

# What questions can Agent mode answer?

Agent mode performs best when you ask specific, data-driven questions about your business data. It handles both straightforward queries and complex questions that need multi-step reasoning, such as:

- Why did revenue spike in June 2025?
- What are our top-selling products this quarter?
- How can we reduce the number of order cancellations this holiday season?
- Tell me how our new product is doing in EMEA
- What customer segments are most profitable?
- What factors contributed to customer churn last quarter?
- Which marketing campaigns had the best ROI and why?

# Frequently asked questions

## Are there additional costs for Agent mode while in Public Preview?

No. During the Public Preview period, Agent mode does not have additional costs beyond the standard warehouse compute costs associated with running SQL queries.

## Does Agent mode use different data than standard Genie?

No. Agent mode uses the same context and data as the standard Genie experience. To learn about the context that Genie uses to generate responses, see How do Genie Spaces generate a response? and Privacy and security.

## Is Agent mode available through the API?

No. During the Public Preview period, users can only submit Agent mode prompts through the Databricks UI.

## What models power Agent mode?

Agent mode uses Sonnet models. Genie is a managed service that continuously evaluates models from multiple providers and uses the most performant and accurate options. For more information, see Databricks AI assistive features trust and safety.

## How do I provide feedback on Agent mode?

Share feedback with your Databricks account team. In particular, Databricks is seeking feedback in the following areas:

- **Research steps**: Whether Agent mode investigates the right dimensions and generates accurate SQL.
- **Answer summaries**: Preferences for format, level of detail, and clarity.
- **Usability**: Any challenges in finding, enabling, or using the feature.
- **Context handling**: Whether Agent mode uses relevant context effectively.
- **Success stories**: Examples where Agent mode delivered valuable insights.

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **May 13, 2026***

# Use the Genie Spaces API

This page explains how to use the Genie API to enable Genie capabilities in your own chatbot, agent, or application.

## Overview

The Genie API provides two types of capabilities:

- **Conversation APIs**: Enable natural language data querying in applications, chatbots, and AI agent frameworks. These APIs support stateful conversations where users can ask follow-up questions and explore data naturally over time.
- **Management APIs**: Enable programmatic creation, configuration, and deployment of Genie Spaces across workspaces. Use these APIs for CI/CD pipelines, version control, and automated space management.

This page describes both conversation and management APIs. Before calling the conversation APIs, prepare a well-curated Genie Space. The space provides the context that Genie uses to interpret questions and generate answers. If the space is incomplete or untested, users might still receive incorrect results even with a correct API integration. This guide explains the minimum setup needed to create a space that works effectively with the Genie API.

The examples on this page use the REST API directly. You can also call these APIs using the Databricks SDKs. See Databricks SDKs.

## Prerequisites

To use the Genie API, you must have:

- Access to a Databricks workspace with the Databricks SQL entitlem

- At least CAN USE privileges on a SQL pro or serverless SQL warehouse.

# Getting started

## Configure Databricks authentication

For production use cases where a user with access to a browser is present, use OAuth for users (OAuth U2M). In situations where browser-based authentication is not possible, use a service principal to authenticate with the API. See OAuth for service principals (OAuth M2M). Service principals must have permissions to access the required data and SQL warehouses.

## Gather details

- **Workspace instance name**: Find and copy your workspace instance name from your Databricks workspace URL. For details about the workspace identifiers in your URL, see Get identifiers for workspace objects.

  Example: `https://cust-success.cloud.databricks.com/`

- **Warehouse ID**: You need the ID of a SQL warehouse that you have at least CAN USE privileges on. To find your warehouse ID:

  i. Go to **SQL Warehouses** in your workspace.
  ii. Select the warehouse you want to use.
  iii. Copy the warehouse ID from the URL or the warehouse details page.

  Alternatively, use the List warehouses endpoint `GET /api/2.0/sql/warehouses` to programmatically retrieve a list of all SQL warehouses that you have permissions to access. The response includes the warehouse ID.

# Create or select a Genie Space

A well-structured Genie Space has the following characteristics:

- **Uses well-annotated data**: Genie relies on table metadata and column comments. Verify that your Unity Catalog data sources have clear, descriptive comments.
- **Is user tested**: Test your space by asking questions you expect from end users. Use testing to create and refine example SQL queries.
- **Includes company-specific context**: Add instructions, example SQL, and functions. See Add SQL examples and instructions. Aim for at least five tested example SQL queries.
- **Uses benchmarks to test accuracy**: Add at least five benchmark questions based on anticipated user questions. See Benchmarks.

For more information on creating a space, see Create and manage a Genie Space and Curate an effective Genie Space.

You can either create a new Genie Space or use an existing one:

**Create a new space** | **Use an existing space**

Create a Genie Space programmatically using the Create Genie Space API. The following example demonstrates a well-structured space that follows best practices. Replace the placeholders with your values:

```
POST /api/2.0/genie/spaces
Host: <DATABRICKS_INSTANCE>
Authorization: Bearer <your_authentication_token>
{
  "description": "Space for analyzing sales performance and trends",
  "parent_path": "/Workspace/Users/<username>",
  "serialized_space": "{\"version\":1,\"config\":{\"sample_questions\":
[{\"id\":\"a1b2c3d4e5f6\",\"question\":[\"What were total sales last month?\
{\"id\":\"b2c3d4e5f6g7\",\"question\":[\"Show top 10 customers by revenue\"]
{\"id\":\"c3d4e5f6g7h8\",\"question\":[\"Compare sales by region for Q1 vs
Q2\"]}]},\"data_sources\":{\"tables\":
[{\"identifier\":\"sales.analytics.orders\",\"description\":[\"Transactional
including order date, amount, and customer information\"],\"column_configs\"
[{\"column_name\":\"order_date\",\"get_example_values\":true},
{\"column_name\":\"status\",\"get_example_values\":true,\"build_value_dictic
{\"column_name\":\"region\",\"get_example_values\":true,\"build_value_dictic
{\"identifier\":\"sales.analytics.customers\"},
{\"identifier\":\"sales.analytics.products\"}]},\"instructions\":{\"text_ins
[{\"id\":\"01f0b37c378e1c91\",\"content\":[\"When calculating revenue, sum t
```

```
column. When asked about 'last month', use the previous calendar month (not
days). Round all monetary values to 2 decimal places.\"]}],\"example_questic
[{\"id\":\"01f0821116d912db\",\"question\":[\"Show top 10 customers by rever
[\"SELECT customer_name, SUM(order_amount) as total_revenue\\n\",\"FROM
sales.analytics.orders o\\n\",\"JOIN sales.analytics.customers c ON o.custom
c.customer_id\\n\",\"GROUP BY customer_name\\n\",\"ORDER BY total_revenue DE
10\"]},{\"id\":\"01f099751a3a1df3\",\"question\":[\"What were total sales la
month\"],\"sql\":[\"SELECT SUM(order_amount) as total_sales\\n\",\"FROM
sales.analytics.orders\\n\",\"WHERE order_date >= DATE_TRUNC('month', CURREN
INTERVAL 1 MONTH)\\n\",\"AND order_date < DATE_TRUNC('month',
CURRENT_DATE)\"]}],\"join_specs\":[{\"id\":\"01f0c0b4e8151\",\"left\":
{\"identifier\":\"sales.analytics.orders\",\"alias\":\"orders\"},\"right\":
{\"identifier\":\"sales.analytics.customers\",\"alias\":\"customers\"},\"sql
[\"orders.customer_id = customers.customer_id\"]}],\"sql_snippets\":{\"filte
[{\"id\":\"01f09972e66d1\",\"sql\":[\"orders.order_amount > 1000\"],\"displa
value orders\",\"synonyms\":[\"large orders\",\"big purchases\"]}],\"express
[{\"id\":\"01f09974563a1\",\"alias\":\"order_year\",\"sql\":
[\"YEAR(orders.order_date)\"],\"display_name\":\"year\"}],\"measures\":
[{\"id\":\"01f09972611f1\",\"alias\":\"total_revenue\",\"sql\":
[\"SUM(orders.order_amount)\"],\"display_name\":\"total revenue\",\"synonyms
[\"revenue\",\"total sales\"]}]}}}",
  "title": "Sales Analytics Space",
  "warehouse_id": "<warehouse-id>"
}

Response:
{
  "space_id": "3c409c00b54a44c79f79da06b82460e2",
  "title": "Sales Analytics Space",
  "description": "Space for analyzing sales performance and trends",
  "warehouse_id": "<warehouse-id>",
  "serialized_space": "{\n  \"version\": 1,\n  \"config\": {\n    \"sample_q
{\n        \"id\": \"a1b2c3d4e5f600000000000000000000\",\n        \"question
\"What were total sales last month?\"\n        ]\n      },\n      {\n
\"b2c3d4e5f6g700000000000000000000\",\n        \"question\": [\n          \"
customers by revenue\"\n        ]\n      },\n      {\n        \"id\":
\"c3d4e5f6g7h800000000000000000000\",\n        \"question\": [\n          \"
by region for Q1 vs Q2\"\n        ]\n      }\n    ]\n  },\n  \"data_sources\
\"tables\": [\n      {\n        \"identifier\": \"sales.analytics.orders\",\
\"description\": [\n          \"Transactional order data including order dat
customer information\"\n        ],\n        \"column_configs\": [\n
\"column_name\": \"order_date\",\n            \"get_example_values\": true\n
{\n            \"column_name\": \"status\",\n            \"get_example_value
\"build_value_dictionary\": true\n          },\n          {\n            \"c
\"region\",\n            \"get_example_values\": true,\n
\"build_value_dictionary\": true\n          }\n        ]\n      },\n      {\
\"identifier\": \"sales.analytics.customers\"\n      },\n      {\n        \"
\"sales.analytics.products\"\n      }\n    ]\n  },\n  \"instructions\": {\n
\"text_instructions\": [\n      {\n        \"id\": \"01f0b37c378e1c91\",\n
\"content\": [\n          \"When calculating revenue, sum the order_amount c
```

```
asked about 'last month', use the previous calendar month (not the last 30 d
monetary values to 2 decimal places.\"\n        ]\n      }\n    ],\n
\"example_question_sqls\": [\n      {\n        \"id\": \"01f0821116d912db\",
\"question\": [\n          \"Show top 10 customers by revenue\"\n        ],\
\"sql\": [\n          \"SELECT customer_name, SUM(order_amount) as total_rev
\"FROM sales.analytics.orders o\\n\",\n          \"JOIN sales.analytics.cust
o.customer_id = c.customer_id\\n\",\n          \"GROUP BY customer_name\\n\"
\"ORDER BY total_revenue DESC\\n\",\n          \"LIMIT 10\"\n        ]\n
\"id\": \"01f099751a3a1df3\",\n        \"question\": [\n          \"What wer
last month\"\n        ],\n        \"sql\": [\n          \"SELECT SUM(order_a
total_sales\\n\",\n          \"FROM sales.analytics.orders\\n\",\n
order_date >= DATE_TRUNC('month', CURRENT_DATE - INTERVAL 1 MONTH)\\n\",\n
order_date < DATE_TRUNC('month', CURRENT_DATE)\"\n        ]\n      }\n    ],
\"join_specs\": [\n      {\n        \"id\": \"01f0c0b4e8151\",\n        \"le
\"identifier\": \"sales.analytics.orders\",\n          \"alias\": \"orders\"
\"right\": {\n          \"identifier\": \"sales.analytics.customers\",\n
\"alias\": \"customers\"\n        },\n        \"sql\": [\n          \"orders
customers.customer_id\"\n        ]\n      }\n    ],\n    \"sql_snippets\": {
\"filters\": [\n        {\n          \"id\": \"01f09972e66d1\",\n          \
[\"orders.order_amount > 1000\"],\n          \"display_name\": \"high value
\"synonyms\": [\"large orders\", \"big purchases\"]\n        }\n      ],\n
\"expressions\": [\n        {\n          \"id\": \"01f09974563a1\",\n
\"order_year\",\n          \"sql\": [\"YEAR(orders.order_date)\"],\n
\"display_name\": \"year\"\n        }\n      ],\n      \"measures\": [\n
\"id\": \"01f09972611f1\",\n          \"alias\": \"total_revenue\",\n
[\"SUM(orders.order_amount)\"],\n          \"display_name\": \"total revenue
\"synonyms\": [\"revenue\", \"total sales\"]\n        }\n      ]\n    }\n  }
}
```


## Understanding the serialized_space field

The `serialized_space` field is a JSON string that defines the configuration and data sources for your Genie Space. In the API request, this JSON must be escaped as a string. The field contains:

- **version**: Schema version number for backwards compatibility. Use `2` as shown in the example below.
- **config**: Space configuration including:
  - **sample_questions**: Example questions to guide users. Each question requires an **id** (32-character hex string) and **question** (array of strings).
- **data_sources**: Data sources available to the space:
  - **tables**: Array of table objects with **identifier** (three-level namespace), optional **description**, and optional **column_configs**.

    - **metric_views**: Array of metric view objects (same structure as tables).
- **instructions**: Structured instructions for the space:
    - **text_instructions**: High-level guidance for the LLM.
    - **example_question_sqls**: Example questions with SQL answers, optionally with **parameters** and **usage_guidance**.
    - **sql_functions**: References to SQL functions available to the space.
    - **join_specs**: Pre-defined join relationships between tables. The `sql` field requires exactly two elements: the join condition, using backtick-quoted alias references, and a relationship type annotation, for example `"--rt=FROM_RELATIONSHIP_TYPE_MANY_TO_ONE--"`. See Join specs format.
    - **sql_snippets**: Reusable **filters**, **expressions**, and **measures**.
- **benchmarks**: Questions for evaluating space quality, each with a ground-truth SQL **answer**.

The unescaped version of the `serialized_space` field from the create space example looks like:

JSON

```
{
  "version": 2,
  "config": {
    "sample_questions": [
      {
        "id": "a1b2c3d4e5f600000000000000000a",
        "question": ["What were total sales last month?"]
      },
      {
        "id": "b2c3d4e5f6a700000000000000000b",
        "question": ["Show top 10 customers by revenue"]
      }
    ]
  },
  "data_sources": {
    "tables": [
      {
        "identifier": "sales.analytics.customers",
        "description": ["Customer master data including contact
information and account details"],
        "column_configs": [
          {
            "column_name": "customer_id",
```

Ask Genie

```
            "description": ["Unique identifier for each customer"],
            "synonyms": ["cust_id", "account_id"]
          },
          {
            "column_name": "customer_name",
            "enable_entity_matching": true
          },
          {
            "column_name": "internal_notes",
            "exclude": true
          }
        ]
      },
      {
        "identifier": "sales.analytics.orders",
        "description": ["Transactional order data including order date,
amount, and customer information"],
        "column_configs": [
          {
            "column_name": "order_date",
            "enable_format_assistance": true
          },
          {
            "column_name": "region",
            "enable_format_assistance": true,
            "enable_entity_matching": true
          },
          {
            "column_name": "status",
            "enable_format_assistance": true,
            "enable_entity_matching": true
          }
        ]
      },
      {
        "identifier": "sales.analytics.products"
      }
    ],
    "metric_views": [
      {
        "identifier": "sales.analytics.revenue_metrics",
        "description": ["Pre-aggregated revenue metrics by region and
time period"],
        "column_configs": [
          {
            "column_name": "period",
            "description": ["Time period for the metric (monthly,
quarterly, yearly)"],
            "enable_format_assistance": true
          }
```

```
        ]
      }
    ]
  },
  "instructions": {
    "text_instructions": [
      {
        "id": "01f0b37c378e1c9100000000000000a1",
        "content": [
          "When calculating revenue, sum the order_amount column. ",
          "When asked about 'last month', use the previous calendar
month. ",
          "Round all monetary values to 2 decimal places."
        ]
      }
    ],
    "example_question_sqls": [
      {
        "id": "01f0821116d912db00000000000000b1",
        "question": ["Show top 10 customers by revenue"],
        "sql": [
          "SELECT customer_name, SUM(order_amount) as total_revenue\n",
          "FROM sales.analytics.orders o\n",
          "JOIN sales.analytics.customers c ON o.customer_id =
c.customer_id\n",
          "GROUP BY customer_name\n",
          "ORDER BY total_revenue DESC\n",
          "LIMIT 10"
        ]
      },
      {
        "id": "01f099751a3a1df300000000000000b2",
        "question": ["What were total sales last month"],
        "sql": [
          "SELECT SUM(order_amount) as total_sales\n",
          "FROM sales.analytics.orders\n",
          "WHERE order_date >= DATE_TRUNC('month', CURRENT_DATE -
INTERVAL 1 MONTH)\n",
          "AND order_date < DATE_TRUNC('month', CURRENT_DATE)"
        ]
      },
      {
        "id": "01f099751a3a1df300000000000000b3",
        "question": ["Show sales for a specific region"],
        "sql": [
          "SELECT SUM(order_amount) as total_sales\n",
          "FROM sales.analytics.orders\n",
          "WHERE region = :region_name"
        ],
        "parameters": [
```

```
          {
            "name": "region_name",
            "type_hint": "STRING",
            "description": ["The region to filter by (e.g., 'North
America', 'Europe')"],
            "default_value": {
              "values": ["North America"]
            }
          }
        ],
        "usage_guidance": ["Use this example when the user asks about
sales filtered by a specific geographic region"]
      }
    ],
    "sql_functions": [
      {
        "id": "01f0c0b4e8151000000000000000f1",
        "identifier": "sales.analytics.fiscal_quarter"
      }
    ],
    "join_specs": [
      {
        "id": "01f0c0b4e8151000000000000000c1",
        "left": {
          "identifier": "sales.analytics.orders",
          "alias": "orders"
        },
        "right": {
          "identifier": "sales.analytics.customers",
          "alias": "customers"
        },
        "sql": ["`orders`.`customer_id` = `customers`.`customer_id`", "--
rt=FROM_RELATIONSHIP_TYPE_MANY_TO_ONE--"],
        "comment": ["Join orders to customers on customer_id"],
        "instruction": ["Use this join when you need customer details for
order analysis"]
      }
    ],
    "sql_snippets": {
      "filters": [
        {
          "id": "01f09972e66d10000000000000000d1",
          "sql": ["orders.order_amount > 1000"],
          "display_name": "high value orders",
          "synonyms": ["large orders", "big purchases"],
          "comment": ["Filters to orders over $1000"],
          "instruction": ["Use when the user asks about high-value or
large orders"]
        }
      ],
```

```
      "expressions": [
        {
          "id": "01f09974563a10000000000000000e1",
          "alias": "order_year",
          "sql": ["YEAR(orders.order_date)"],
          "display_name": "year",
          "synonyms": ["fiscal year", "calendar year"],
          "comment": ["Extracts the year from order date"],
          "instruction": ["Use for year-over-year analysis"]
        }
      ],
      "measures": [
        {
          "id": "01f09972611f10000000000000000f1",
          "alias": "total_revenue",
          "sql": ["SUM(orders.order_amount)"],
          "display_name": "total revenue",
          "synonyms": ["revenue", "total sales"],
          "comment": ["Sum of all order amounts"],
          "instruction": ["Use this measure for revenue calculations"]
        }
      ]
    }
  },
  "benchmarks": {
    "questions": [
      {
        "id": "01f0d0b4e81510000000000000000g1",
        "question": ["What is the average order value?"],
        "answer": [
          {
            "format": "SQL",
            "content": ["SELECT AVG(order_amount) as avg_order_value\n",
"FROM sales.analytics.orders"]
          }
        ]
      }
    ]
  }
}
```

When constructing your space, create this JSON structure and then escape it as a string for the API request. For complete schema details, see the Create Genie Space API reference.

## Validation rules for serialized_space

The `serialized_space` JSON must conform to the following validation rules. JSON that is not valid is rejected during space creation or update.

## Version

- **Version field**: Required. Use `2` for new spaces. The version number exists for backwards compatibility.

## ID format

All ID fields must be **32-character lowercase hexadecimal strings** (UUID format without hyphens).

- **Valid**: `a1b2c3d4e5f6000000000000000000a`
- **Not valid**: `a1b2c3d4e5f6` (too short), `A1B2C3D4E5F6000000000000000000A` (uppercase), `a1b2c3d4-e5f6-0000-0000-00000000000a` (contains hyphens)

IDs are required for:

- `config.sample_questions[].id`
- `instructions.text_instructions[].id`
- `instructions.example_question_sqls[].id`
- `instructions.join_specs[].id`
- `instructions.sql_snippets.filters[].id`
- `instructions.sql_snippets.expressions[].id`
- `instructions.sql_snippets.measures[].id`
- `benchmarks.questions[].id` (if benchmarks are included)

You can use the following command to generate a valid ID:

Bash

```
python3 -c "import random,datetime;t=int((datetime.datetime.now()-
datetime.datetime(1582,10,15)).total_seconds()*1e7);print(f'{(t&0xFFFFFFFFF
(1<<12)|((t&0xFFFF)>>4):016x}{random.getrandbits(62)|0x8000000000000000:016x
```

This generates a time-ordered UUID. IDs generated in sequence sort alphabetically in the order they were created, which satisfies the sorting requirements automatically.

## Sorting requirements

Collections containing IDs or identifiers must be pre-sorted. The system validates that arrays are already sorted and rejects unsorted input.

| Collection | Sort key |
|---|---|
| `data_sources.tables` | `identifier` (alphabetically) |
| `data_sources.metric_views` | `identifier` (alphabetically) |
| `data_sources.tables[].column_configs` | `column_name` (alphabetically) |
| `data_sources.metric_views[].column_configs` | `column_name` (alphabetically) |
| `config.sample_questions` | `id` (alphabetically) |
| `instructions.text_instructions` | `id` (alphabetically) |
| `instructions.example_question_sqls` | `id` (alphabetically) |
| `instructions.sql_functions` | `(id, identifier)` tuple (alphabetically) |
| `instructions.join_specs` | `id` (alphabetically) |
| `instructions.sql_snippets.filters` | `id` (alphabetically) |
| `instructions.sql_snippets.expressions` | `id` (alphabetically) |
| `instructions.sql_snippets.measures` | `id` (alphabetically) |
| `benchmarks.questions` | `id` (alphabetically) |

## Uniqueness constraints

- **Question IDs**: All IDs in `config.sample_questions` and `benchmarks.questions` must be unique across both collections.

- **Instruction IDs**: All IDs across `text_instructions`, `example_question_sqls`, `sql_functions`, `join_specs`, and all `sql_snippets` types must be unique.
- **Column configs**: The combination of `(table_identifier, column_name)` must be unique within the space.

## Size and length limits

- **String length**: Individual string elements are limited to 25,000 characters.
- **Array size**: Repeated fields are limited to 10,000 items.
- **Text instructions**: At most 1 text instruction is allowed per space.
- **Tables and metric views**: Subject to workspace-specific limits.
- **SQL content**: Query text in `sql` and `join_specs.sql` fields is subject to length limits.

## Join specs format

The `sql` field in each join spec must contain **exactly two elements**:

1. The join condition, using backtick-quoted alias references:

   Text

   ```
   "`orders`.`customer_id` = `customers`.`customer_id`"
   ```

2. A relationship type annotation in the following format:

   Text

   ```
   "--rt=FROM_RELATIONSHIP_TYPE_<CARDINALITY>--"
   ```

   Valid cardinality values:

   - `FROM_RELATIONSHIP_TYPE_MANY_TO_ONE`
   - `FROM_RELATIONSHIP_TYPE_ONE_TO_MANY`
   - `FROM_RELATIONSHIP_TYPE_ONE_TO_ONE`
   - `FROM_RELATIONSHIP_TYPE_MANY_TO_MANY`

Omitting the relationship type annotation causes the API to reject the request with a parsing error. For multi-column joins, create a separate join spec for each relationship.

### Other requirements

- **Table identifiers**: Must use three-level namespace format (`catalog.schema.table`).
- **Benchmark answers**: Each benchmark question must have exactly one answer with format set to SQL.
- **SQL snippets**: Filter, expression, and measure SQL fields must not be empty.

## Using the conversation API

After you configure a Genie Space, use the conversation API endpoints to ask questions, retrieve results, and maintain multi-turn conversations with context.

### Start a conversation

The Start conversation endpoint `POST /api/2.0/genie/spaces/{space_id}/start-conversation` starts a new conversation in your Genie Space.

Replace the placeholders with your Databricks instance, Genie Space ID, and authentication token. An example of a successful response follows the request. It includes details that you can use to access this conversation again for follow-up questions.

```
POST /api/2.0/genie/spaces/{space_id}/start-conversation

HOST= <DATABRICKS_INSTANCE>
Authorization: <your_authentication_token>
{
    "content": "<your question>",
}


Response:

{
  "conversation": {
```

```
    "created_timestamp": 1719769718,
    "id": "6a64adad2e664ee58de08488f986af3e",
    "last_updated_timestamp": 1719769718,
    "space_id": "3c409c00b54a44c79f79da06b82460e2",
    "title": "Give me top sales for last month",
    "user_id": 12345
  },
  "message": {
    "attachments": null,
    "content": "Give me top sales for last month",
    "conversation_id": "6a64adad2e664ee58de08488f986af3e",
    "created_timestamp": 1719769718,
    "error": null,
    "id": "e1ef34712a29169db030324fd0e1df5f",
    "last_updated_timestamp": 1719769718,
    "query_result": null,
    "space_id": "3c409c00b54a44c79f79da06b82460e2",
    "status": "IN_PROGRESS",
    "user_id": 12345
  }
}
```


## Retrieve generated SQL

Use the `conversation_id` and `message_id` in the response to poll to check the message's generation status and retrieve the generated SQL from Genie. See `GET /api/2.0/genie/spaces/{space_id}/conversations/{conversation_id}/messages/{message_id}` for complete request and response details.

> **NOTE**
>
> Only `POST` requests count toward the queries-per-minute throughput considerations. `GET` requests used to poll results are not subject to this limit.

Substitute your values into the following request:

```
GET
/api/2.0/genie/spaces/{space_id}/conversations/{conversation_id}/messages/{m
HOST= <DATABRICKS_INSTANCE>
Authorization: Bearer <your_authentication_token>
```



The following example response reports the message details:

```
Response:

{
  "attachments": null,
  "content": "Give me top sales for last month",
  "conversation_id": "6a64adad2e664ee58de08488f986af3e",
  "created_timestamp": 1719769718,
  "error": null,
  "id": "e1ef34712a29169db030324fd0e1df5f",
  "last_updated_timestamp": 1719769718,
  "query_result": null,
  "space_id": "3c409c00b54a44c79f79da06b82460e2",
  "status": "IN_PROGRESS",
  "user_id": 12345
}
```

The `attachments` field is progressively populated during processing. When the status is `PENDING_WAREHOUSE` or `EXECUTING_QUERY`, you can already begin reading the `attachments` field. The response populates incrementally, starting with the generated SQL query, followed by the description, follow-up questions, and additional context. The `COMPLETED` status indicates that the reply process has fully finished and polling can stop, but meaningful content is available earlier if your client inspects the response during polling rather than waiting for completion.

To determine whether a response was generated using a trusted asset, check the `attachments` field in the response for a `query.parameters` object. Its presence indicates the answer came from a trusted asset.

To access Genie's reasoning traces, check the `attachments` field for a `query_attachments` object of type `GenieQueryAttachments`. When present, it contains the step-by-step reasoning Genie used to generate the response. For complete field details, see the Get message API reference.

## Retrieve query results

The `attachments` array contains Genie's response. It includes the generated text response (`text`), the query statement if it exists (`query`), and an identifier that you can use to get the associated query results (`attachment_id`). Replace the placeholders in the following example to retrieve the generated query results.

> **NOTE**
>
> The Genie Conversation API returns tabular query results as structured data. It does not return rendered charts or visualizations. To display charts, retrieve the query results from the `attachment_id` and render them in your application using a charting library of your choice. The `query` field in the attachment contains the SQL that Genie generated, which you can also run directly against your warehouse to retrieve results for visualization.

> **NOTE**
>
> Unlike the Genie Spaces UI, the Genie Conversation API does not support the two-phase response pattern where Genie shows a preliminary answer and then updates it after inspection. To surface incremental progress to users, inspect the `attachments` field while polling during `PENDING_WAREHOUSE` or `EXECUTING_QUERY` states rather than waiting for `COMPLETED`.

```
GET
/api/2.0/genie/spaces/{space_id}/conversations/{conversation_id}/messages/{m
result/{attachment_id}
Authorization: Bearer <your_authentication_token>
```

See `GET /api/2.0/genie/spaces/{space_id}/conversations/{conversation_id}/messages/{message_id}/attachments/{attachment_id}/query-result`.

## Ask follow-up questions

After you receive a response, use the `conversation_id` to continue the conversation. Context from previous messages is retained and used in follow-up responses. For complete request and response details, see `POST /api/2.0/genie/spaces/{space_id}/conversations/{conversation_id}/messages`.

```
POST
/api/2.0/genie/spaces/{space_id}/conversations/{conversation_id}/messages
HOST= <DATABRICKS_INSTANCE>
Authorization: <your_authentication_token>
{
```

```
  "content": "Which of these customers opened and forwarded the email?",
}
```

## Add comments to messages

You can add text comments to messages and list existing comments using the message comment API endpoints.

To add a comment to a message, use the Create message comment endpoint `POST /api/2.0/genie/spaces/{space_id}/conversations/{conversation_id}/messages/{message_id}/comments`:

```
POST
/api/2.0/genie/spaces/{space_id}/conversations/{conversation_id}/messages/{m
HOST= <DATABRICKS_INSTANCE>
Authorization: Bearer <your_authentication_token>
{
  "content": "<your comment text>"
}
```

To list existing comments on a message, use the List message comments endpoint.

# Retrieve space and conversation data

The Genie API provides additional endpoints for retrieving configuration and historical data from existing spaces and conversations.

## Retrieve space configuration

When retrieving space information using the Get Genie Space API, you can include the `serialized_space` field in the response by setting the `include_serialized_space` parameter to `true`. The `serialized_space` field contains the serialized string representation of the Genie Space, including instructions, benchmarks, joins, and other configuration details.

Use this serialized representation with the Create Genie Space API and Update Genie Space API to promote Genie Spaces across workspaces or create backups of space

configurations.

Example GET request:

```
GET /api/2.0/genie/spaces/{space_id}?include_serialized_space=true
Host: <DATABRICKS_INSTANCE>
Authorization: Bearer <your_authentication_token>

Response:
{
  "space_id": "3c409c00b54a44c79f79da06b82460e2",
  "title": "Sales Analytics Space",
  "description": "Space for analyzing sales performance and trends",
  "warehouse_id": "<warehouse-id>",
  "serialized_space": "{\"version\":1,\"config\":{\"sample_questions\":
[{\"id\":\"a1b2c3d4e5f600000000000000000000\",\"question\":[\"What were tota
month?\"]},{\"id\":\"b2c3d4e5f6g700000000000000000000\",\"question\":[\"Show
customers by revenue\"]}]},\"data_sources\":{\"tables\":
[{\"identifier\":\"sales.analytics.orders\",\"description\":[\"Transactional
including order date, amount, and customer information\"],\"column_configs\"
[{\"column_name\":\"order_date\",\"get_example_values\":true},
{\"column_name\":\"status\",\"get_example_values\":true,\"build_value_dictic
{\"column_name\":\"region\",\"get_example_values\":true,\"build_value_dictic
{\"identifier\":\"sales.analytics.customers\"},
{\"identifier\":\"sales.analytics.products\"}]},\"instructions\":{\"text_ins
[{\"id\":\"01f0b37c378e1c91\",\"content\":[\"When calculating revenue, sum t
column. When asked about 'last month', use the previous calendar month (not
days). Round all monetary values to 2 decimal places.\"]}],\"example_questic
[{\"id\":\"01f0821116d912db\",\"question\":[\"Show top 10 customers by rever
[\"SELECT customer_name, SUM(order_amount) as total_revenue\\n\",\"FROM
sales.analytics.orders o\\n\",\"JOIN sales.analytics.customers c ON o.custom
c.customer_id\\n\",\"GROUP BY customer_name\\n\",\"ORDER BY total_revenue DE
10\"]},{\"id\":\"01f099751a3a1df3\",\"question\":[\"What were total sales la
month\"],\"sql\":[\"SELECT SUM(order_amount) as total_sales\\n\",\"FROM
sales.analytics.orders\\n\",\"WHERE order_date >= DATE_TRUNC('month', CURREN
INTERVAL 1 MONTH)\\n\",\"AND order_date < DATE_TRUNC('month',
CURRENT_DATE)\"]}],\"join_specs\":[{\"id\":\"01f0c0b4e8151\",\"left\":
{\"identifier\":\"sales.analytics.orders\",\"alias\":\"orders\"},\"right\":
{\"identifier\":\"sales.analytics.customers\",\"alias\":\"customers\"},\"sql
[\"orders.customer_id = customers.customer_id\"]}],\"sql_snippets\":{\"filte
[{\"id\":\"01f09972e66d1\",\"sql\":[\"orders.order_amount > 1000\"],\"displa
value orders\",\"synonyms\":[\"large orders\",\"big purchases\"]}],\"express
[{\"id\":\"01f09974563a1\",\"alias\":\"order_year\",\"sql\":
[\"YEAR(orders.order_date)\"],\"display_name\":\"year\"}],\"measures\":
[{\"id\":\"01f09972611f1\",\"alias\":\"total_revenue\",\"sql\":
[\"SUM(orders.order_amount)\"],\"display_name\":\"total revenue\",\"synonyms
```

```
[\"revenue\",\"total sales\"]}]}}}"
}
```

## Reference old conversation threads

To allow users to refer to old conversation threads, use the List conversation messages endpoint `GET /api/2.0/genie/spaces/{space_id}/conversations/{conversation_id}/messages` to retrieve all messages from a specific conversation thread.

## Retrieve conversation data for analysis

Space managers can programmatically retrieve all previous messages asked across all users of a space for analysis. To retrieve this data:

1. Use the `GET /api/2.0/genie/spaces/{space_id}/conversations` endpoint to get all existing conversation threads in a space.
2. For each conversation ID returned, use the `GET /api/2.0/genie/spaces/{space_id}/conversations` endpoint to retrieve the list of messages for that conversation.

# Best practices and limits

## Best practices for using the Genie API

To maintain performance and reliability when using the Genie API:

- **Implement retry logic with exponential backoff**: The API doesn't retry failed requests for you, so add your own queuing and exponential backoff. This helps your application handle transient failures, avoid unnecessary repeat requests, and stay within throughput limits as it grows.
- **Log API responses**: Implement comprehensive logging of API requests and responses to help with debugging, monitoring usage patterns, and tracking costs.
- **Poll for status updates every 1 to 5 seconds**: Continue polling until a conclusive message status, such as `COMPLETED`, `FAILED`, or `CANCELLED`, is received. Ask Genie

polling to 10 minutes for most queries. If there is no conclusive response after 10 minutes, stop polling and return a timeout error or prompt the user to manually check the query status later.

- **Use exponential backoff for polling**: Increase the delay between polls up to a maximum of one minute. This reduces unnecessary requests for long-running queries while still allowing low latency for fast ones.
- **Start a new conversation for each session**: Avoid reusing conversation threads across sessions, as this can reduce accuracy due to unintended context reuse.
- **Maintain conversation limits**: To manage old conversations and stay under the 10,000 conversation limit:
  i. Use the `GET /api/2.0/genie/spaces/{space_id}/conversations` endpoint to see all existing conversation threads in a space.
  ii. Identify conversations that are no longer needed, such as older conversations or test conversations.
  iii. Use the `DELETE /api/2.0/genie/spaces/{space_id}/conversations/{conversation_id}` endpoint to remove conversations programmatically.

## Throughput considerations

Throughput rates for the Genie conversation API free tier are best-effort and depend on system capacity. To mitigate misuse and prevent abuse during peak usage periods, the system processes requests based on available capacity. Under normal or low-traffic conditions, the API supports requests to five questions per minute per workspace. If you're seeking higher throughput support, contact your Databricks account team.

# Monitor the space

After your application is set up, you can monitor questions and responses in the Databricks UI.

Encourage users to test the space so that you learn about the types of questions they are likely to ask and the responses they receive. Provide users with guidance to help them start testing the space. Use the **Monitoring** tab to view questions and responses. See Monitor the space.

You can also use audit logs to monitor activity in a Genie Space. See Genie Space events.

**this page**

**as this page helpful?**

Yes | No

Send feedback